

Relatório do Projeto

Jogo da Velha em Rede com Sockets TCP

Disciplina: Redes de Computadores

Aplicação cliente-servidor desenvolvida em Python

Aluno: Luiz Henrique Lima Leite

Ano/Semestre: 2026.1

Este documento apresenta o relatório técnico da aplicação de Jogo da Velha implementada em Python, utilizando comunicação em rede por sockets TCP e processamento concorrente com threads. O relatório descreve as principais funcionalidades, os protocolos utilizados, o que poderia ter sido implementado a mais, as dificuldades encontradas durante o desenvolvimento e, ao final, inclui a impressão do código-fonte no mesmo arquivo.

1. Introdução

O projeto consiste em uma versão em rede do jogo da velha. A aplicação foi organizada no modelo cliente-servidor: um programa servidor fica responsável por controlar a partida e dois programas clientes se conectam a ele para realizar as jogadas. Essa estrutura permite demonstrar, de forma prática, conceitos importantes de Redes de Computadores, como sockets, conexão TCP, troca de mensagens, controle de estado no servidor e comunicação entre processos em máquinas diferentes.

O jogo utiliza uma comunicação textual simples. O cliente envia ao servidor a posição escolhida pelo jogador, e o servidor valida a jogada, atualiza o tabuleiro, verifica se houve vencedor ou empate e envia o novo estado da partida aos participantes. Desse modo, a lógica principal fica centralizada no servidor, evitando que cada cliente tenha uma visão diferente do jogo.

2. Objetivo da aplicação

O objetivo principal foi implementar uma aplicação distribuída simples, mas funcional, capaz de permitir que dois jogadores disputem uma partida de jogo da velha utilizando comunicação por rede. A proposta também busca aplicar na prática o uso da biblioteca socket da linguagem Python, o modelo cliente-servidor e o uso de threads para lidar com comunicação simultânea.

- Permitir a conexão de dois jogadores ao servidor.
- Atribuir automaticamente os símbolos X e O aos jogadores.
- Controlar a alternância de turnos durante a partida.
- Validar jogadas inválidas, fora do intervalo ou em posições já ocupadas.
- Atualizar e enviar o tabuleiro para os clientes após cada jogada.
- Identificar vitória, empate e encerramento da partida.

3. Principais funcionalidades implementadas

3.1 Servidor

O servidor é o componente central da aplicação. Ele cria um socket TCP, associa esse socket ao endereço 0.0.0.0 e à porta 5050, e fica aguardando conexões de clientes. A cada conexão aceita, o servidor atribui um símbolo ao jogador: o primeiro recebe X e o segundo recebe O. Quando os dois jogadores estão conectados, a partida é iniciada.

Durante o jogo, o servidor mantém o tabuleiro em uma lista com nove posições, registra qual jogador tem o turno atual e utiliza uma função de verificação para identificar linhas, colunas ou diagonais vencedoras. Caso todas as posições sejam preenchidas sem vencedor, o resultado é tratado como empate.

3.2 Cliente

O cliente tem a função de se conectar ao servidor, receber mensagens da partida e enviar a entrada digitada pelo usuário. Ao iniciar, o usuário pode informar o IP do servidor ou pressionar Enter para usar o endereço local 127.0.0.1. Após a conexão, o cliente inicia uma thread dedicada ao recebimento de mensagens, permitindo que o jogador continue digitando comandos enquanto mensagens do servidor são exibidas na tela.

3.3 Regras do jogo

As regras implementadas seguem a dinâmica tradicional do jogo da velha. O tabuleiro possui nove posições numeradas de 1 a 9. O jogador somente pode jogar quando for sua vez, e a posição escolhida precisa estar livre. A vitória ocorre quando um jogador completa uma linha,

coluna ou diagonal com seu símbolo. O empate ocorre quando o tabuleiro fica completo sem combinação vencedora.

4. Protocolos e conceitos de rede utilizados

A aplicação utiliza o protocolo TCP como base da comunicação entre cliente e servidor. No código, isso é representado pela criação de sockets com a família AF_INET, para comunicação IPv4, e o tipo SOCK_STREAM, que indica o uso de uma conexão orientada a fluxo, característica do TCP.

- IPv4: usado como família de endereçamento dos sockets, por meio de socket.AF_INET.
- TCP: usado por meio de socket.SOCK_STREAM, fornecendo comunicação orientada à conexão.
- Porta 5050: porta escolhida para o servidor aguardar conexões dos clientes.
- Protocolo de aplicação simples: as mensagens trocadas são textos codificados e decodificados entre cliente e servidor.

A escolha do TCP é adequada para esse tipo de jogo porque as mensagens precisam chegar de forma confiável e na ordem correta. Em uma partida de jogo da velha, perder uma jogada ou receber jogadas fora de ordem causaria inconsistência no estado do tabuleiro. Por isso, o TCP é mais apropriado que o UDP para esta implementação.

5. Arquitetura da solução

Arquivo	Responsabilidade
server.py	Gerencia conexões, jogadores, tabuleiro, turnos, validação de jogadas, vitória, empate e envio de mensagens.
client.py	Conecta ao servidor, recebe mensagens em uma thread e envia entradas digitadas pelo usuário.
README.md	Explica os requisitos, comandos de execução, uso de dois clientes e funcionamento básico da partida.

A arquitetura adotada segue o modelo centralizado. O servidor é a autoridade sobre o estado da partida; os clientes não decidem o resultado do jogo, apenas enviam as escolhas dos jogadores e exibem as respostas recebidas. Essa abordagem facilita a validação das regras e reduz conflitos de estado entre os participantes.

6. Funcionamento da execução

Para executar a aplicação, primeiro deve-se iniciar o servidor. Em seguida, dois clientes devem ser abertos em terminais separados. O primeiro cliente conectado será o jogador X e o segundo será o jogador O. Depois que os dois entram na partida, o servidor mostra o tabuleiro e solicita a jogada do jogador da vez.

```
python server.py  
python client.py
```

Se todos os programas estiverem na mesma máquina, o cliente pode utilizar localhost. Caso os clientes estejam em outra máquina da mesma rede, deve-se informar o IP da máquina que está executando o servidor.

7. O que poderia ser implementado a mais

Embora a aplicação cumpra o objetivo principal, existem melhorias que poderiam ampliar a robustez, a usabilidade e a organização do projeto. Entre as principais possibilidades estão:

- Interface gráfica para substituir a interação apenas por terminal.

- Sistema de nomes de jogadores, placar e histórico de partidas.
- Suporte a várias salas simultâneas, permitindo mais de uma partida ao mesmo tempo.
- Tratamento mais completo para desconexão inesperada de um jogador durante a partida.
- Opção de reiniciar a partida sem precisar reconectar os clientes.
- Protocolo de aplicação mais estruturado, com comandos como JOIN, MOVE, BOARD, WIN e ERROR.
- Uso de logs para registrar conexões, jogadas, erros e encerramento de partidas.
- Configuração da porta e do IP por argumentos de linha de comando ou arquivo de configuração.
- Camada de segurança com TLS em uma versão mais avançada.

8. Dificuldades encontradas

Uma das principais dificuldades foi organizar a comunicação entre dois clientes de forma que ambos recebessem atualizações corretas do tabuleiro. Como o jogo depende da ordem das jogadas, o servidor precisa controlar rigorosamente o turno atual e impedir que um jogador envie uma jogada fora da sua vez.

Outra dificuldade foi lidar com concorrência. O servidor cria uma thread para cada jogador, o que facilita o atendimento simultâneo, mas exige cuidado para que duas threads não modifiquem o tabuleiro ao mesmo tempo. Para reduzir esse problema, foi utilizado um lock, garantindo que o bloco de validação e atualização da jogada seja executado de forma protegida.

Também foi necessário validar entradas do usuário, como números fora do intervalo de 1 a 9, caracteres não numéricos e posições já ocupadas. Além disso, a finalização da partida precisou fechar as conexões dos clientes e limpar o estado do servidor para evitar inconsistências.

9. Conclusão

O projeto demonstrou de maneira prática como uma aplicação simples pode ser distribuída em rede usando o modelo cliente-servidor. A implementação com TCP foi adequada porque o jogo exige confiabilidade, ordem na entrega das mensagens e consistência entre os jogadores. O uso de threads tornou a aplicação mais responsiva, permitindo que o servidor lide com mais de uma conexão e que o cliente receba mensagens sem bloquear completamente a entrada do usuário.

Portanto, o trabalho atende ao objetivo de implementar uma aplicação de rede funcional e relaciona diretamente conceitos estudados em Redes de Computadores, como sockets, portas, conexões TCP, troca de mensagens, concorrência e arquitetura cliente-servidor.

Anexo A - Impressão do código-fonte

A seguir está a impressão do código-fonte utilizado no projeto, conforme solicitado nas especificações do relatório. Foram incluídos os arquivos principais do servidor e do cliente.

A.1 Código-fonte do servidor

Arquivo: **server.py**

```
001: import socket
002: import threading
003:
004: HOST = '0.0.0.0'
005: PORT = 5050
006:
007: board = [' ']*9
008: players = [] # lista de (conn, addr, simbolo)
009: lock = threading.Lock()
010: current_turn = 0 # índice em players de quem deve jogar
011:
012:
013: def reset_board():
014:     global board, current_turn
015:     board = [' ']*9
016:     current_turn = 0
017:
018:
019: def render_board():
020:     b = board
021:     return (
022:         f"\n {b[0]} | {b[1]} | {b[2]} \n"
023:         f"---+---+---\n"
024:         f" {b[3]} | {b[4]} | {b[5]} \n"
025:         f"---+---+---\n"
026:         f" {b[6]} | {b[7]} | {b[8]} \n"
027:     )
028:
029:
030: def check_winner():
031:     wins = [
032:         (0,1,2),(3,4,5),(6,7,8), # linhas
033:         (0,3,6),(1,4,7),(2,5,8), # colunas
034:         (0,4,8),(2,4,6) # diagonais
035:     ]
036:     for a,b,c in wins:
037:         if board[a] == board[b] == board[c] != ' ':
038:             return board[a]
039:     if ' ' not in board:
040:         return 'empate'
041:     return None
042:
043:
044: def broadcast(msg, exclude=None):
045:     for conn, addr, sym in players:
046:         if conn != exclude:
047:             try:
048:                 conn.sendall(msg.encode())
049:             except:
050:                 pass
051:
052:
053: def send_to(conn, msg):
054:     try:
055:         conn.sendall(msg.encode())
056:     except:
057:         pass
058:
059:
060: def handle_client(conn, addr, player_index):
061:     global current_turn
062:
063:     sym = players[player_index][2]
064:     print(f"[CONEXÃO] {addr} conectado como {sym}")
065:
066:     # Espera o segundo jogador
067:     send_to(conn, f"Você é o jogador {sym}. Aguardando oponente...\n")
068:
069:     while len(players) < 2:
070:         pass # espera ativa simples
071:
072:     send_to(conn, "Jogo iniciado!\n")
073:     send_to(conn, render_board())
074:
075:     if player_index == 0:
076:         send_to(conn, "Sua vez! Digite a posição (1-9): ")
077:     else:
078:         send_to(conn, "Aguarde o oponente jogar...\n")
079:
080:     while True:
081:         try:
082:             data = conn.recv(1024).decode().strip()
083:         except:
084:             break
085:
086:         if not data:
```

```
087:         break
088:
089:     with lock:
090:         # Verifica se é a vez deste jogador
091:         if current_turn != player_index:
092:             send_to(conn, "Não é sua vez! Aguarde.\n")
093:             continue
094:
095:         # Valida entrada
096:         if not data.isdigit() or not (1 <= int(data) <= 9):
097:             send_to(conn, "Posição inválida! Digite um número de 1 a 9: ")
098:             continue
099:
100:         pos = int(data) - 1
101:         if board[pos] != ' ':
102:             send_to(conn, "Posição ocupada! Escolha outra: ")
103:             continue
104:
105:         # Faz a jogada
106:         board[pos] = sym
107:
108:         # Envia tabuleiro atualizado para todos
109:         board_str = render_board()
110:         broadcast(board_str)
111:
112:     winner = check_winner()
113:     if winner:
114:         if winner == 'empate':
115:             broadcast("Empate! Nenhum vencedor.\n")
116:         else:
117:             broadcast(f"Jogador {winner} venceu!\n")
118:             broadcast("Encerrando conexão. Obrigado por jogar!\n")
119:             # Fecha conexões após pequeno delay
120:             for c, a, s in players:
121:                 try:
122:                     c.close()
123:                 except:
124:                     pass
125:             reset_board()
126:             players.clear()
127:             return
128:
129:     # Passa a vez
130:     current_turn = 1 - current_turn
131:     next_conn = players[current_turn][0]
132:     send_to(next_conn, "Sua vez! Digite a posição (1-9): ")
133:     other = players[1 - current_turn][0]
134:     send_to(other, "Aguarde o oponente jogar...\n")
135:
136: print(f"[DESCONEÇÃO] {addr}")
137:
138:
139: def start_server():
140:     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
141:     server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
142:     server.bind((HOST, PORT))
143:     server.listen(2)
144:     print(f"[SERVIDOR] Aguardando jogadores em {HOST}:{PORT}...")
145:
146:     while True:
147:         conn, addr = server.accept()
148:         if len(players) >= 2:
149:             conn.sendall("Partida em andamento. Tente mais tarde.\n".encode())
150:             conn.close()
151:             continue
152:
153:         sym = 'X' if len(players) == 0 else 'O'
154:         players.append((conn, addr, sym))
155:         idx = len(players) - 1
156:
157:         t = threading.Thread(target=handle_client, args=(conn, addr, idx))
158:         t.daemon = True
159:         t.start()
160:
161:
162: if __name__ == '__main__':
163:     start_server()
```

A.2 Código-fonte do cliente

Arquivo: **client.py**

```
001: import socket
002: import threading
003: import sys
004:
005: HOST = '127.0.0.1'
006: PORT = 5050
007:
008:
009: def receive_messages(conn):
010:     """Thread que fica escutando mensagens do servidor e imprime na tela."""
011:     while True:
012:         try:
013:             msg = conn.recv(1024).decode()
014:             if not msg:
015:                 print("\n[INFO] Conexão encerrada pelo servidor.")
016:                 break
017:             print(msg, end='', flush=True)
018:         except:
019:             print("\n[INFO] Conexão perdida.")
020:             break
021:
022:
023: def start_client():
024:     host = input("IP do servidor (Enter para localhost): ").strip()
025:     if not host:
026:         host = HOST
027:
028:     conn = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
029:     try:
030:         conn.connect((host, PORT))
031:     except Exception as e:
032:         print(f"Não foi possível conectar: {e}")
033:         sys.exit(1)
034:
035:     print("Conectado ao servidor!\n")
036:
037:     # Thread para receber mensagens do servidor
038:     t = threading.Thread(target=receive_messages, args=(conn,))
039:     t.daemon = True
040:     t.start()
041:
042:     # Loop principal: lê entrada do usuário e envia ao servidor
043:     while True:
044:         try:
045:             entrada = input()
046:             conn.sendall(entrada.encode())
047:         except (EOFError, KeyboardInterrupt):
048:             print("\nSaindo...")
049:             conn.close()
050:             break
051:     except:
052:         break
053:
054:
055: if __name__ == '__main__':
056:     start_client()
```

Anexo B - Orientações de execução

Este anexo resume as instruções de execução descritas no README do projeto.

Jogo da Velha - Redes de Computadores 2026.1

Aplicação cliente-servidor de Jogo da Velha implementada em Python utilizando sockets TCP e threads.

Requisitos

- Python 3.x instalado
- Dois terminais (ou dois computadores na mesma rede)

Como rodar

1. Iniciar o servidor

Abra um terminal e execute:

```
python server.py
```

O servidor ficará aguardando dois jogadores na porta **5050**.

2. Conectar os clientes

Abra **dois terminais separados** (podem ser na mesma máquina ou em máquinas diferentes na mesma rede) e execute em cada um:

```
python client.py
```

- Se estiver rodando tudo na mesma máquina, pressione **Enter** quando perguntar o IP (usará localhost automaticamente).
- Se o cliente estiver em outra máquina, informe o IP da máquina onde o servidor está rodando (ex: `192.168.1.10`).

3. Jogar

- O primeiro a conectar será **X**, o segundo será **O**.
- O tabuleiro usa posições de **1** a **9**, conforme o mapa abaixo:

```
1 | 2 | 3
---+---
4 | 5 | 6
---+---
7 | 8 | 9
```

- Quando for sua vez, digite o número da posição e pressione **Enter**.
- O jogo termina quando há um vencedor ou empate.

Arquitetura

Arquivo	Descrição
`server.py`	Servidor TCP que gerencia o estado do jogo
`client.py`	Cliente que se conecta ao servidor e interage

- **Protocolo**: TCP via sockets (`socket.AF_INET`, `socket.SOCK_STREAM`)
- **Threads**: o servidor cria uma thread por jogador; o cliente usa uma thread dedicada para receber mensagens do servidor sem bloquear a entrada.